

Object-Oriented Parking System Design Document

The purpose of this project was to develop a parking system that is capable of registration of customers, associates vehicles with said customers, and then issues permits. The application does need a way to maintain information regarding the customers and cars, while still providing a simple way to register. A single customer could have several vehicles, and each would need differentiating information such as the plate and type. There is also a requirement that the system allows for registration of compact vs SUV type vehicles as it affects pricing and discounts.

The implementation focus was on creating a basic structure that has scalability available as the parking system expands (parking is added). Instead of putting all of the application functionality into a single class, the application separates responsibilities between domain objects, the parking office, and the command-processing layer. This results in easier implementation and testing.

System Design and Implementation

The UML diagram included (bottom), represents classes and relations implemented into the Parking Application. The main domain classes are Customer, Car, Address, and CarType. Customer is responsible for storing a customer's identifiable information like number, name, phone number, and address. Address stores the actual address, and Customer can be associated with registered vehicles in the system.

Car as a class represents the individual vehicle, not a collection of vehicles a user may own. It stores the vehicle's CarType, license plate, and owner. CarType is an enumeration comprising the two car types Compact and SUV.

ParkingOffice is responsible for housing and maintaining registered customers and their vehicles. It uses maps to store cars and owners, using identifiers as keys. This allows the application to retrieve a customer or vehicle when its identifier is known. It also maintains counters used to generate the customer ID's and permit numbers.

ParkingService is responsible for the main interface for processing application commands. It has reference to the ParkingOffice and maintains a map of commands. The current implementation creates the commands: RegisterCustomerCommand and RegisterCarCommand.

Command interface establishes a common structure for commands by defining methods for getting command name, display names, and executing the actual command.

RegisterCustomerCommand and RegisterCarCommand each validate their required parameters before using the ParkingOffice to perform the executed operation.

Last, ParkingApplication provides the main() method used to demo the system functionality. It creates required objects, completes registration, and displays results.

The project is standardized on **Java 17**. Maven is used to manage the project and execute the tests. The project uses **JUnit 5.10.2** for automated unit testing.

Unit Test

The project includes testing that is automated and covers the primary function of the implemented classes. The test confirm basic behavior of the Customer and Car classes. Including user information, car type, plate, and owner relationship. Testing also covers the registration functionality provided by ParkingService and ParkingOffice. User registration is tested and is verified that a customer can be registered and assigned an identifier. Vehicle registration is also tested to verify that a registered vehicle gets a permit identifier.

The test suite contains 16 tests, with 0 failures, 0 errors, and 0 skipped tests. The tests were executed using Maven with the following command:

```
mvn test
```

The resulting Maven build completed successfully with a BUILD SUCCESS result.

Application Execution and Results

The application entry point is the ParkingApplication class. Running this classes demos the registration workflow that is implemented by the project.

Expected output:

Customer registered successfully: customer-1

Car registered successfully; permit issued: permit-1

This output shows that the application can successfully create customer registration, then registration a car, then issue a permit.

Challenges, Lessons Learned, and Conclusion

Hands down, the biggest challenge for me this week was getting back into Java. For the last year I have worked at a web-marketing company, mostly focused on TypeScript, React, Node, and a lot of webstack. Some of the syntax and general operations required some review and a lot of refreshers. Getting back into classes and Maven based testing took longer than I would have preferred, but this is just the way things are going to go.

Working the project came with a lot of moments where I was reminded of what I was doing and how things came together. Interestingly Java is where I started coding and it was at times frustrating seeing how much I was struggling at first.

Another insightful part of the project was working with interfaces and the command structure. The Command interface allowed the registration commands to follow the same structure while keeping behaviors separate.

Overall, this was a solid reintroduction into Java and object-oriented development, while also producing a semi-functional application.

example

parkingsystem

